유비샘 세미나 추가 내용

유비샘 2차 세미나

유비샘 3차 세미나

유비샘 4차 세미나

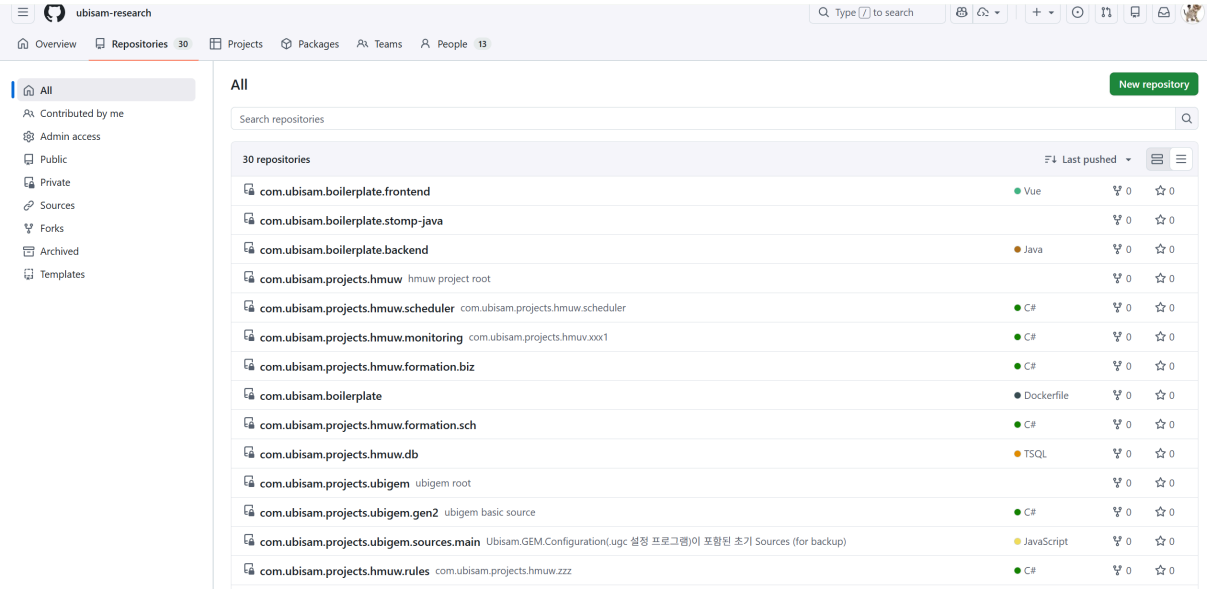
> 💡 이번 내용은 위 3개의 세미나 관련 추가 내용입니다.

COC

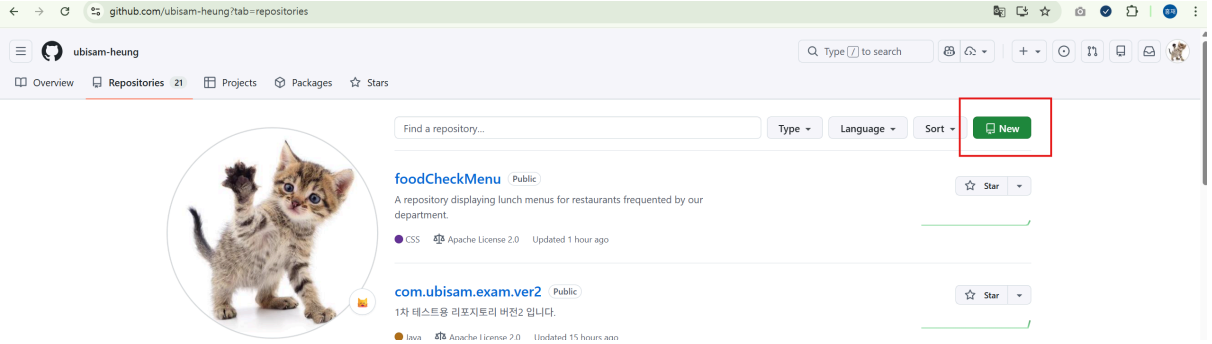
git에서 리포지토리 생성과 클론

- github에서 리포지토리를 생성하실 때 아래 링크와 사진을 토대로 규칙이 있는걸 확인하실 수 있습니다.

https://github.com/orgs/ubisam-research/repositories



- 해당 규칙대로 리포지토리를 생성해보겠습니다.



- 세미나 코드 백엔드 리포지토리를 생성하기 위해 규칙을 정합니다.

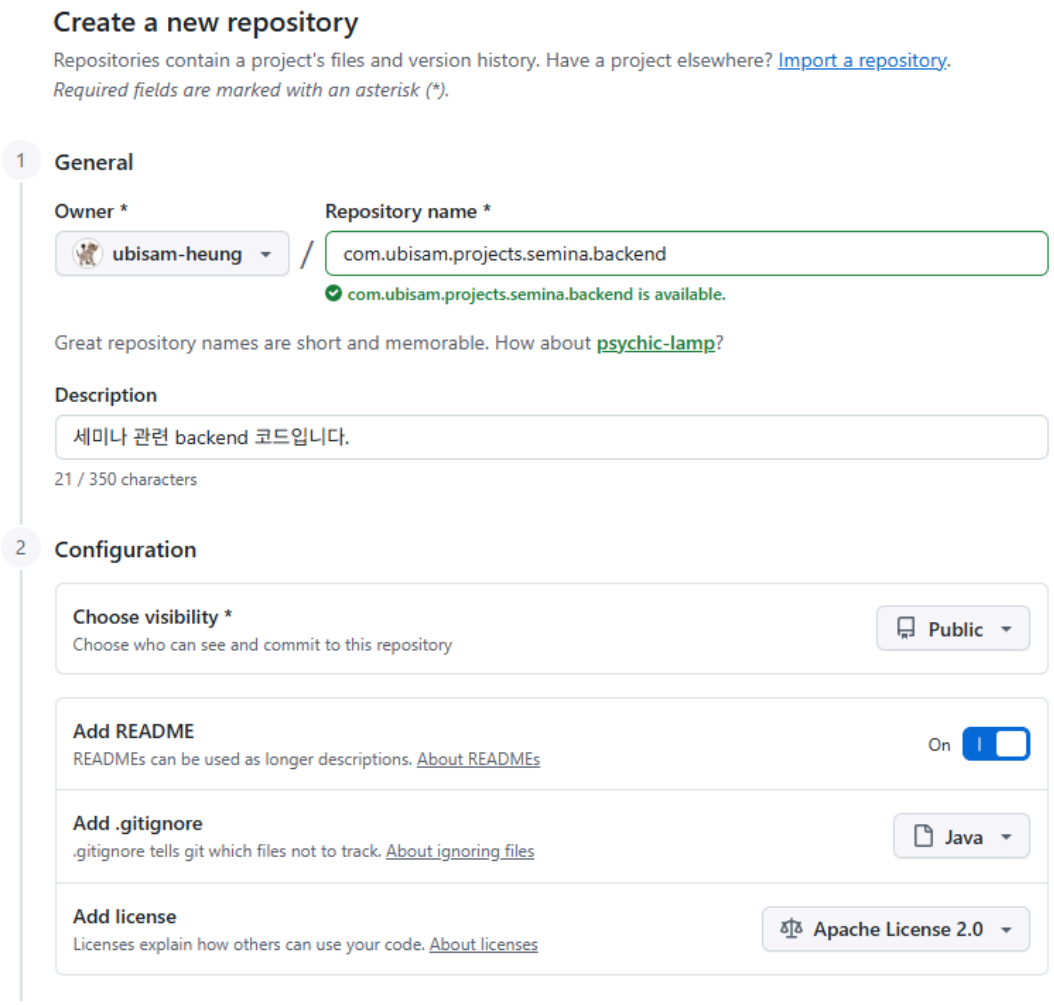
  com.ubisam.projects.semina.backend

  com.ubisam.projects는 ubisam-research의 규칙을 따릅니다.

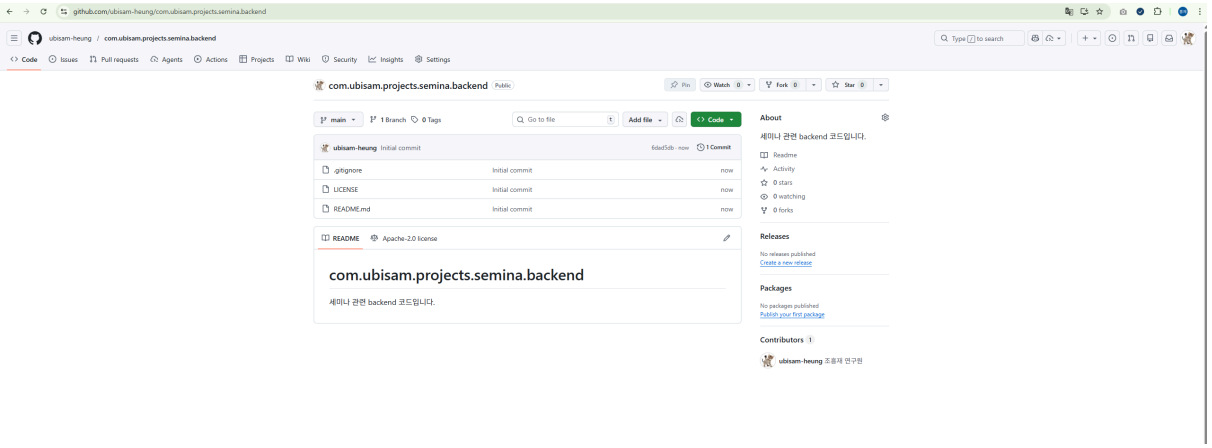
  세미나 관련 코드이기에 .semina 를 붙여줍니다.

  백엔드 코드이기에 .backend를 붙여줍니다.

- 규칙대로 생성 후 아래 사진과 같이 설정해줍니다.



- 아래 사진과 같이 생성된 모습을 확인할 수 있습니다.



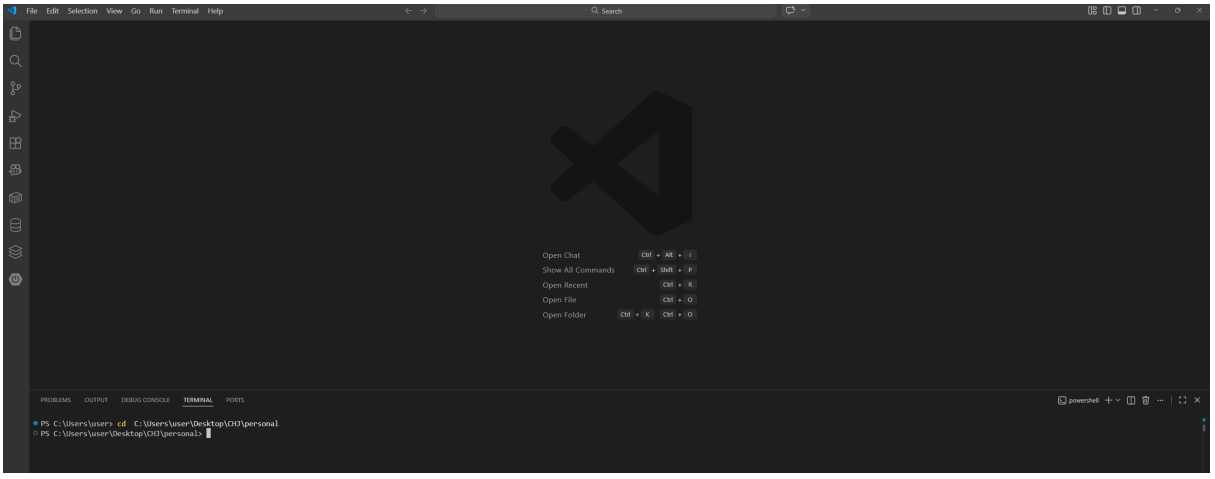
- 해당 리포지토리를 clone 해보겠습니다.

  clone의 위치는 자신의 개인 폴더 밑에 clone 해주시면됩니다.

  제 개인 폴더 위치는 C:\Users\user\Desktop\CHJ\personal 입니다.

- VSCode를 열어 터미널을 여신 후 cd [개인 폴더 위치]로 이동합니다.

  아래 사진과 같이 이동된걸 확인하실 수 있습니다.



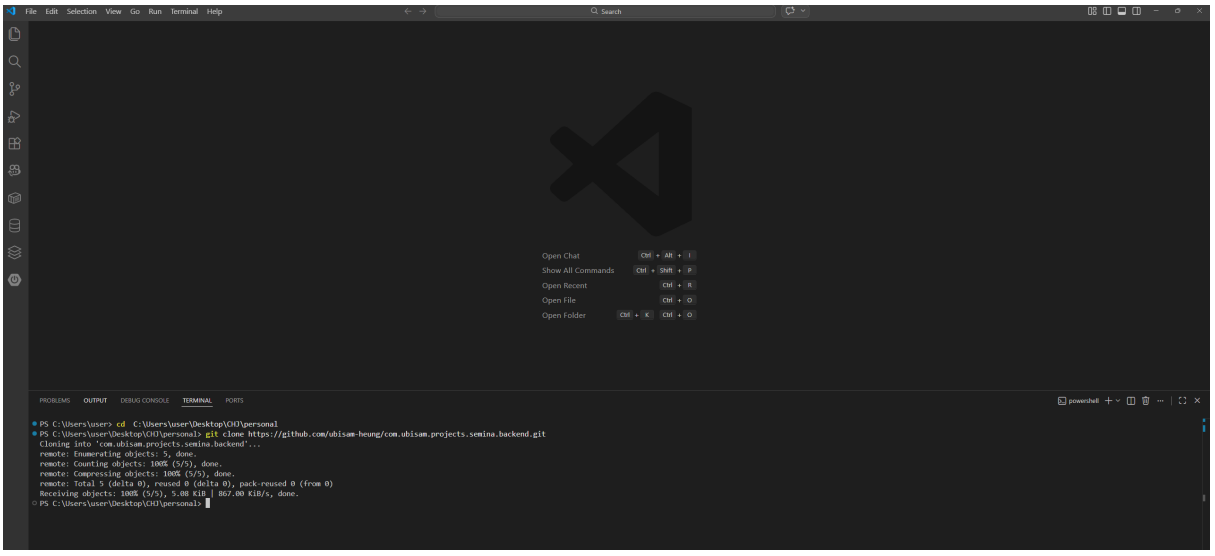
- 그 후 해당 위치에서 git clone [만든 리포지토리] 명령어를 입력해줍니다.

  제 기준으로는

  git clone https://github.com/ubisam-heung/com.ubisam.projects.semina.backend.git

  입니다.

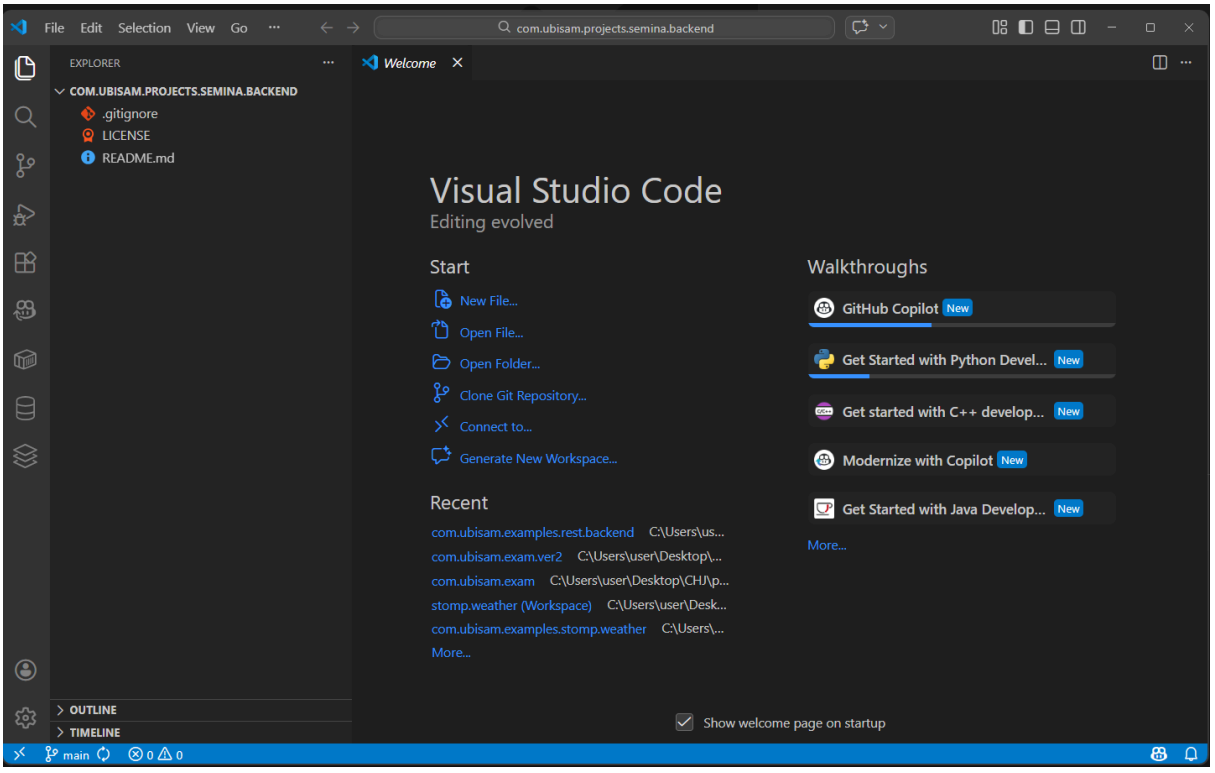
- 아래 사진과 같이 정상적으로 clone이 완료 되었습니다.



- cd com.ubisam.projects.semina.backend 로 해당 폴더 내부로 이동합니다.

```
Receiving objects: 100% (5/5), 5.08 KiB | 867.00 KiB/s, done.
PS C:\Users\user\Desktop\CHJ\personal> cd com.ubisam.projects.semina.backend
PS C:\Users\user\Desktop\CHJ\personal\com.ubisam.projects.semina.backend>
```

- 그 후 code . 명령어를 통해 새 창으로 해당 폴더를 엽니다.



- 이제 이 내부에서 코드 작업을 수행하시면 됩니다.
- Frontend도 동일하게 수행하시면 됩니다.

  그 후 이전에 배웠던 workspace를 추가하여 backend와 frontend를 동시에 작업하게끔 설정합니다.

  최종적으로 아래 사진과 같이 설정되면 됩니다.

  - 내부 코드 작업은 따로 이전 세미나 코드를 옮기시면 됩니다.

Test Code

- 현재 저희 테스트 코드는 ApplicationTests.java에 전부 적혀있는 방식입니다.

  해당 방식은 COC를 위배하기 때문에 변경해주어야 합니다.
- 먼저 현재 폴더 구조 방식을 살펴보겠습니다.

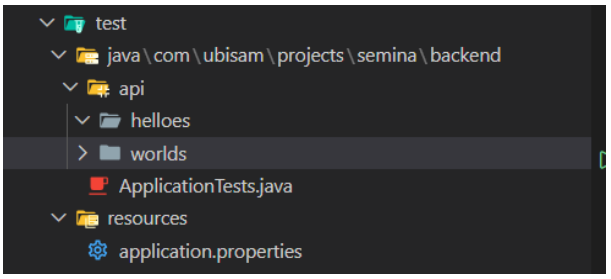
```
com.ubisam.projects.semina.backend/
├── mvnw, mvnw.cmd, pom.xml         # 프로젝트 설정
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/ubisam/projects/semina/backend/
│   │   │       ├── Application.java            # Spring B
oot 메인 클래스
│   │   │       ├── ApplicationApiConfig.java   # API 설정
클래스
│   │   │       ├── ApplicationStompConfig.java # Stomp 설
정 클래스
│   │   │       ├── api/                        # REST API
컨트롤러 계층
│   │   │       ├── domain/                     # 도메인(엔
티티,리포지토리, 서비스)
│   │   │       └── stomp/                      # Stomp 코
드
│   │   └── resources/
│   │       └── application.properties          # 환경 및 D
B 설정
```

- 제 backend 코드 기준으로는 이런식으로 설정되어있습니다.

- 저희는
  - Hello에 대한 테스트,
  - World에 대한 테스트

  를 만들어주어야 합니다.

- test 폴더 밑에 backend 폴더 밑에 api 폴더를 생성해줍니다.
- 그 후 api 폴더 밑에 저희 코드 기준으로 helloes폴더와 worlds 폴더를 생성해줍니다.



- helloes 폴더 밑에 HelloTests.java 를 생성해줍니다.

  그 후 ApplicationTests.java에 contextLoad2 구간을 복사하여 붙여넣습니다.

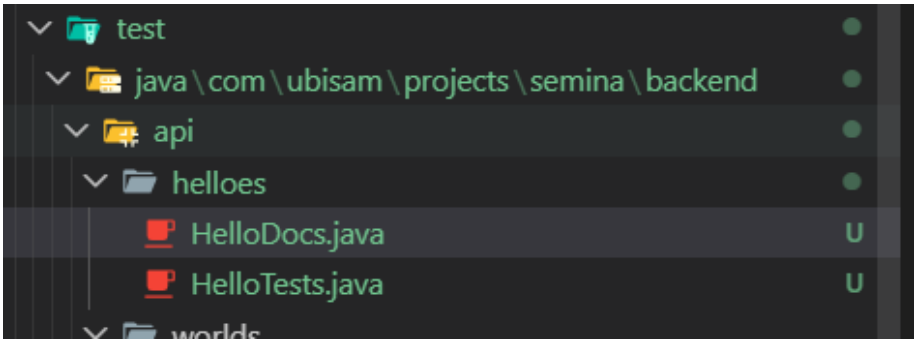
  아래와 같이 코드가 작성됩니다.

```
import static io.u2ware.common.docs.MockMvcRestDocs.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.boot.test.autoconfigure.web.serv
let.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTes
t;
import org.springframework.test.web.servlet.MockMvc;

import com.ubisam.projects.semina.backend.domain.Hello;

@SpringBootTest
@AutoConfigureMockMvc
public class HelloTests {

  @Autowired
  private MockMvc mockMvc;
```

```
    @Test
    void contextLoad() throws Exception {
        //perform : 요청 구간
        //andDo : 응답 구간(결과가 나오고 나서 무언가를 하고 싶을
때)
        //andExpect : 검증 구간(결과를 기대하는 구간)

        Hello h = new Hello();
        h.setName("name1");
        h.setEmail("abc@abc.com");

        //Create
        mockMvc.perform(post("/api/helloes").content(h)).an
dDo(print()).andExpect(is2xx());

        //Read
        // mockMvc.perform(get("/api/helloes")).andDo(print
()).andExpect(is2xx());
        // mockMvc.perform(get("/helloes/"+ h.getId())).and
Do(print()).andExpect(is2xx());

        // h.setName("name2");
        // h.setEmail("abc1@abc1.com");

        //Update
        // mockMvc.perform(put("/helloes/1").content(h)).an
dDo(print()).andExpect(is2xx());

        //Delete
        // mockMvc.perform(delete("/helloes/1").content
(h)).andDo(print()).andExpect(is2xx());

        //Read
        // mockMvc.perform(get("/helloes")).andDo(print()).
andExpect(is2xx());
    }
}
```

- Hello h 로 매번 선언 후 하드코딩으로 객체를 만드는것은 매우 비효율적입니다.

  그래서 helloes 폴더 밑에 HelloDocs.java 파일을 생성해 줍니다.



- HelloDocs.java 안에는 아래와 같이 코딩해줍니다.

```
import java.util.HashMap;
import java.util.Map;

import org.springframework.stereotype.Component;

import io.u2ware.common.docs.MockMvcRestDocs;

@Component
public class HelloDocs extends MockMvcRestDocs{

  public Map<String, Object> newEntity(String name){
    Map<String, Object> hello = new HashMap<>();
    hello.put("name", name);
    hello.put("email", super.randomText("") + "@" + super.r
andomText("") + ".com");
    return hello;
  }

  public Map<String, Object> updateEntity(Map<String, Objec
t> entity, String name){
    entity.put("name", name);
    return entity;
  }

  public Map<String, Object> setKeyword(String keyword){
```

```
    Map<String, Object> hello = new HashMap<>();
    hello.put("keyword", keyword);
    return hello;
  }
}
```

- newEntity는 name을 매개변수로 받아옵니다.

  즉, 사용자가 name을 입력하면 해당 내용을 받아와서

  객체 생성 시 name 부분에 사용자 입력값을 넣어주고,

  email에는 “랜덤한텍스트@랜덤한텍스트.com” 형식으로 만드는 코드입니다.

  randomText("")의 “” 안에는 prefix라고 해서 접두사를 붙여줄 수 있습니다.
- updateEntity는 entity와 name을 매개변수로 받아옵니다.

  즉, entity는 변환할 entity, name은 사용자의 입력값이 들어갑니다.

  그 후 사용자 입력값으로 “name” 을 다시 세팅합니다.
- setKeyword는 RestfulJpaRepository를 사용하기에 넣어준 코드입니다.

  사용자로부터 검색용 키워드를 받아와서 “keyword”에 넣습니다.

- 이제 HelloTest.java를 수정합니다.

```
        Hello h = new Hello();
        h.setName("name1");
        h.setEmail("abc@abc.com");
```

- 저희는 위 코드 부분이 필요없으므로 제거합니다.

  그 후 아래 코드를 추가하여 HelloDocs.java를 받아옵니다.

```
@Autowired
private HelloDocs docs;
```

```java
@SpringBootTest
@AutoConfigureMockMvc
public class HelloTests {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private HelloDocs docs;

    @Test
    void contextLoad() throws Exception {
        //perform : 요청 구간
        //andDo : 응답 구간(결과가 나오고 나서 무언가를 하고 싶을때)
        //andExpect : 검증 구간(결과를 기대하는 구간)

        //Create
        mockMvc.perform(post(uri: "/api/helloes").content(h)).andDo(print()).andExpect(is2xx());
```

- Create 부분부터 변경해보겠습니다.

  아래 코드로 변경합니다.

```
mockMvc
    .perform(post("/api/helloes")
    .content(docs::newEntity, "김길동"))
    .andDo(print())
    .andExpect(is2xx())
    .andDo(result(docs::context , "entity1"))
;
```

- content에 받아온 docs의 newEntity를 사용하여 사용자 입력값으로 “김길동”을 넣습니다.

  저희가 입력한 “김길동”은 name에 들어갑니다. 그 후 print()를 출력합니다.

  그리고 결과값을 entity1 이라는 “key” 값으로 저장합니다.

- 마지막 줄을 위해 ApplicationApiConfig.java로 이동합니다.

  아래 코드를 CORS 설정 위에 추가합니다.

```
// POST, PUT 요청에 대한 응답으로 변경된 엔티티 반환 설정
config.setReturnBodyForPutAndPost(true);
```

- 아래 사진과 같이 코드를 작성합니다.

```java
import io.u2ware.common.data.jpa.config.EnableRestfulJpaRepositories;

@Configuration
@EnableRestfulJpaRepositories
@EnableJpaRepositories
public class ApplicationApiConfig implements RepositoryRestConfigurer {


    @Override
    public void configureRepositoryRestConfiguration(RepositoryRestConfiguration config, CorsRegistry cors) {
        // /api 주소 설정
        config.setBasePath(basePath: "/api");
        // id 노출 설정
        config.exposeIdsFor(...domainTypes: Hello.class, World.class);
        // POST, PUT 요청에 대한 응답으로 변경된 엔티티 반환 설정
        config.setReturnBodyForPutAndPost(returnBody: true);
        // CORS 설정 - 모든 주소 허용, 모든 메소드 허용, 쿠키 허용 안함, maxAge 설정
        cors.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("*")
            .allowCredentials(false)
            .maxAge(999999);
    }
}
```

- 테스트를 실행하시면 아래와 같이 정상적으로 실행됨을 확인하실 수 있습니다.

  Hibernate가 insert를 실행하고 입력한 값인 김길동도 정상적으로 들어갑니다.

```
Hibernate:
    call next value for example_hello_seq
Hibernate:
    insert
    into
        example_hello
        (email, name, id)
    values
        (?, ?, ?)
2026-03-13T11:49:59.498+09:00 TRACE 17076 --- [example2] [          main] org.hibernate.orm.jdbc.bind             : binding parameter (1:VARCHAR) <- [37@34.com]
2026-03-13T11:49:59.499+09:00 TRACE 17076 --- [example2] [          main] org.hibernate.orm.jdbc.bind             : binding parameter (2:VARCHAR) <- [김길동]
2026-03-13T11:49:59.499+09:00 TRACE 17076 --- [example2] [          main] org.hibernate.orm.jdbc.bind             : binding parameter (3:BIGINT) <- [1]

http://localhost/api/helloes
----------------------------------------

MockHttpServletRequest:
      HTTP Method = POST
      Request URI = /api/helloes
       Parameters = {}
          Headers = [Content-Type:"application/json;charset=UTF-8"]
             Body = {"name":"김길동","email":"37@34.com"}
    Session Attrs = {}

Handler:
             Type = org.springframework.data.rest.webmvc.RepositoryEntityController
           Method = org.springframework.data.rest.webmvc.RepositoryEntityController#postCollectionResource(RootResourceInformation, PersistentEntityResource, Persist
```

- Debug Console에 MockHttpServletResponse 부분을 보시면 아래 내용이 출력되는 것을 확인하실 수 있습니다.
  - config.setReturnBodyForPutAndPost(true);

    설정에 의해 Body가 출력됩니다.

```
Body = {
 "id" : 1,
```

```
  "name" : "김길동",
  "email" : "71@96.com",
  "keyword" : null,
  "helloId" : null,
  "_links" : {
    "self" : {
      "href" : "http://localhost/api/helloes/1"
    },
    "hello" : {
      "href" : "http://localhost/api/helloes/1"
    }
  }
}
```

- 위 내용이 entity1이라는 key값으로 value에 저장되어있습니다.

  이제 김길동에 대한 엔티티에 대해 Read, Update, Delete를 해보겠습니다.

- Read 부분입니다. CRUD에서 Read는 단건 행만 읽어오는 것을 뜻합니다.

  이전 코드에서는 객체에서 id값을 출력하여 print하거나 하드코딩으로 id값을 넣어줘서 가져왔지만

  이젠 entity1에서 url값을 가져와서 Read를 합니다.

  아래 코드로 변경해줍니다.

```
    String url = docs.context("entity1", "$._links.self.hre
f");
    mockMvc
      .perform(post(url))
      .andDo(print())
      .andExpect(is2xx())
    ;
```

```java
//Create
mockMvc
    .perform(post(uri: "/api/helloes")
    .content(docs::newEntity, t: "김길동"))
    .andDo(print())
    .andExpect(is2xx())
    .andDo(result(docs::context , key: "entity1"))
;

//Read
String url = docs.context(key: "entity1", jsonPath: "$._links.self.href");
mockMvc
    .perform(post(url))
    .andDo(print())
    .andExpect(is2xx())
;
```

- url에는 entity1에 _links.self.href 값을 저장합니다.

  위 코드 기준으론 http://localhost/api/helloes/1 가 url에 들어갑니다.

- RestfulJpaRepository 는 Read를 할 때 JpaRepository와 다르게 POST를 사용합니다.
  - JpaRepository는 GET을 사용

  먼저 잘 읽어오는지 테스트를 실행하여 초록색 마크가 뜨는지 확인합니다.

- 아래 사진을 통해 정상적으로 Hibernate가 select 문을 실행한다는 것을 확인해보실 수 있습니다.

```
Hibernate:
    select
        h1_0.id,
        h1_0.email,
        h1_0.name
    from
        example_hello h1_0
    where
        h1_0.id=?
2026-03-13T12:01:50.789+09:00 TRACE 17444 --- [example2] [           main] org.hibernate.orm.jdbc.bind              : binding parameter (1:BIGINT) <- [1]

http://localhost/api/helloes/1
----------------------------------------

MockHttpServletRequest:
      HTTP Method = POST
      Request URI = /api/helloes/1
       Parameters = {}
          Headers = [Content-Type:"application/json;charset=UTF-8"]
             Body = {}
    Session Attrs = {}
```

- 그럼 RestfulJpaRepository에서 get으로 불러오면 무슨 결과가 일어나는지 테스트해보겠습니다.

  아래 코드를 추가합니다.

```
mockMvc
  .perform(get(url))
  .andDo(print())
  .andExpect(is4xx())
;
```

```
//Create
mockMvc
  .perform(post(uri: "/api/helloes")
  .content(docs::newEntity, t: "김길동"))
  .andDo(print())
  .andExpect(is2xx())
  .andDo(result(docs::context , key: "entity1"))
;

//Read
String url = docs.context(key: "entity1", jsonPath: "$._links.self.href");
mockMvc
  .perform(post(url))
  .andDo(print())
  .andExpect(is2xx())
;
mockMvc
  .perform(get(url))
  .andDo(print())
  .andExpect(is4xx())
;
```

- url는 위에 선언했으니 넣으면 되며, isExpect는 2xx이 아닌 4xx를 넣어줘서 실패할 것이라는걸 기대합니다.

  그 후 테스트를 실행하여 초록색 마크를 확인합니다.

- 아래 사진을 통해 get을 요청하였을때 405가 반환되는 것을 확인해보실 수 있습니다.

  즉, 저희가 4xx 번대 Status가 오는 것을 기대했고 실제로 4xx번대 코드가 왔으니 테스트는 성공입니다.

```
MockHttpServletRequest:
      HTTP Method = GET
      Request URI = /api/helloes/1
       Parameters = {}
          Headers = []
             Body = null
     Session Attrs = {}
```

```
MockHttpServletResponse:
           Status = 405
    Error message = null
          Headers = [Vary:"Origin", "Access-Contr
     Content type = null
             Body =
    Forwarded URL = null
   Redirected URL = null
          Cookies = []
```

- Update 부분입니다. Update는 PUT 메소드를 사용합니다.

  저희는 이전 Docs에 Update부분을 updateEntity로 만들었습니다.

  아래 코드로 변경합니다.

```
Map<String,Object> body = docs.context("entity1");
mockMvc
  .perform(put(url)
  .content(docs::updateEntity, body, "박길동"))
  .andExpect(is2xx())
  .andDo(print())
;
```

```
//Update
Map<String,Object> body = docs.context(key: "entity1");
mockMvc
  .perform(put(url)
  .content(docs::updateEntity, body, u: "박길동"))
  .andExpect(is2xx())
  .andDo(print())
;
```

- updateEntity에게는 entity와 name을 전달해주어야 합니다.

  그렇기에 Map 형태로 body에 저장 후 바꿀 이름인 “박길동” 이랑 같이 넣습니다.

  테스트를 실행하여 초록색 마크를 확인합니다.

- 아래 사진을 통해 Hibernate가 update를 요청하였고, PUT으로 보낸걸 확인해 보실 수 있습니다.

```
Hibernate:
    update
        example_hello
    set
        email=?,
        name=?
    where
        id=?
2026-03-13T13:26:19.676+09:00 TRACE 22116 --- [example2] [           main] org.hibernate.orm.jdbc.bind
2026-03-13T13:26:19.676+09:00 TRACE 22116 --- [example2] [           main] org.hibernate.orm.jdbc.bind
2026-03-13T13:26:19.677+09:00 TRACE 22116 --- [example2] [           main] org.hibernate.orm.jdbc.bind

http://localhost/api/helloes/1
-----------------------------------------

MockHttpServletRequest:
      HTTP Method = PUT
      Request URI = /api/helloes/1
       Parameters = {}
          Headers = [Content-Type:"application/json;charset=UTF-8"]
             Body = {"id":1,"name":"박길동","email":"85@14.com","keyword":null,"helloId":null,"_links":{
    Session Attrs = {}

Handler:
```

- 또한 응답도 200으로 2xx번대 코드가 온 것을 확인해 보실 수 있습니다.

```
MockHttpServletResponse:
           Status = 200
    Error message = null
          Headers = [Vary:"Origin", "Access-Control-Request-Method", "Access-Control-Request-Headers", Location:"http://localhost/api/helloes/1",
     Content type = application/hal+json
             Body = {
  "id" : 1,
  "name" : "박길동",
  "email" : "85@14.com",
  "keyword" : null,
  "helloId" : null,
  "_links" : {
    "self" : {
      "href" : "http://localhost/api/helloes/1"
    },
    "hello" : {
      "href" : "http://localhost/api/helloes/1"
    }
```

- Delete 부분입니다. Delete는 DELETE 메소드를 사용합니다.

  entity1에 대해 delete를 실행해보기위해 아래 코드로 변경합니다.

  그 후 테스트를 실행하여 초록색 마크를 확인합니다.

```
mockMvc
    .perform(delete(url))
    .andDo(print())
    .andExpect(is2xx())
;
```

- 아래 사진을 통해 Hibernate가 delete를 호출한 것을 확인할 수 있고 Method는 DELETE로 요청한 것을

  확인해 보실 수 있습니다.

```
Hibernate:
    delete
    from
        example_hello
    where
        id=?
2026-03-13T13:30:25.689+09:00 TRACE 1368 --- [example2] [           main] org.hibernate.orm.jdbc.bind             : binding parameter (1:BIGINT) <- [1]

http://localhost/api/helloes/1
----------------------------------------

MockHttpServletRequest:
      HTTP Method = DELETE
      Request URI = /api/helloes/1
       Parameters = {}
          Headers = []
             Body = null
    Session Attrs = {}

Handler:
             Type = org.springframework.data.rest.webmvc.RepositoryEntityController
           Method = org.springframework.data.rest.webmvc.RepositoryEntityController#deleteItemResource(RootResourceInformation, Serializable, ETag, Persis
```

- 최종적으로 HelloTests.java 코드는 아래와 같습니다.

```
import static io.u2ware.common.docs.MockMvcRestDocs.*;


import java.util.Map;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.boot.test.autoconfigure.web.serv
let.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTes
t;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
public class HelloTests {

  @Autowired
  private MockMvc mockMvc;
```

```
  @Autowired
  private HelloDocs docs;

  @Test
    void contextLoad() throws Exception {
        //perform : 요청 구간
        //andDo : 응답 구간(결과가 나오고 나서 무언가를 하고 싶을
때)
        //andExpect : 검증 구간(결과를 기대하는 구간)

        //Create
    mockMvc
      .perform(post("/api/helloes")
      .content(docs::newEntity, "김길동"))
      .andDo(print())
      .andExpect(is2xx())
      .andDo(result(docs::context , "entity1"))
    ;

        //Read
    String url = docs.context("entity1", "$._links.self.hre
f");
    mockMvc
      .perform(post(url))
      .andDo(print())
      .andExpect(is2xx())
    ;
    mockMvc
      .perform(get(url))
      .andDo(print())
      .andExpect(is4xx())
    ;

        //Update
    Map<String,Object> body = docs.context("entity1");
        mockMvc
      .perform(put(url)
```

```
      .content(docs::updateEntity, body, "박길동"))
      .andExpect(is2xx())
      .andDo(print())
    ;

        //Delete
    mockMvc
      .perform(delete(url))
      .andDo(print())
      .andExpect(is2xx())
    ;
    }
}
```

- 추가로 JpaRepository와 RestfulJpaRepository 차이점을 테스트하는 코드를 추가하겠습니다.

  해당 코드는

  - world - JpaRepository
  - hello - RestfulJpaRepository

  로 테스트 코드를 작성하겠습니다.

- 먼저 test 폴더 밑에 있는 worlds 폴더 밑에 HelloDocs.java를 복사하여

  WorldDocs.java로 변경한 후 추가합니다.

  코드는 아래와 같이 작성합니다.

```
import java.util.HashMap;
import java.util.Map;

import org.springframework.stereotype.Component;

import io.u2ware.common.docs.MockMvcRestDocs;

@Component
public class WorldDocs extends MockMvcRestDocs{

  public Map<String, Object> newEntity(String name){
```

```
    Map<String, Object> world = new HashMap<>();
    world.put("name", name);
    world.put("email", super.randomText("") + "@" + super.r
andomText("") + ".com");
    return world;
  }

  public Map<String, Object> updateEntity(Map<String, Objec
t> entity, String name){
    entity.put("name", name);
    return entity;
  }
}
```

```
package com.ubisam.projects.semina.backend.api.worlds;

import java.util.HashMap;
import java.util.Map;

import org.springframework.stereotype.Component;

import io.u2ware.common.docs.MockMvcRestDocs;

@Component
public class WorldDocs extends MockMvcRestDocs{

  public Map<String, Object> newEntity(String name){
    Map<String, Object> world = new HashMap<>();
    world.put("name", name);
    world.put("email", super.randomText(prefix: "") + "@" + super.randomText(prefix:
    return world;
  }

  public Map<String, Object> updateEntity(Map<String, Object> entity, String name){
    entity.put("name", name);
    return entity;
  }
}
```

- 다음으로 HelloTests.java를 복사하여 WorldTests.java로 변환 후 추가합니다.

  아래와 같은 코드를 작성합니다.

  엔티티 명은 김길동, 박길동 그대로 사용하겠습니다.

```
import static io.u2ware.common.docs.MockMvcRestDocs.*;


import java.util.Map;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowir
ed;
import org.springframework.boot.test.autoconfigure.web.serv
let.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTes
t;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
public class WorldTests {

  @Autowired
  private MockMvc mockMvc;

  @Autowired
  private WorldDocs docs;

  @Test
    void contextLoad() throws Exception {
        //perform : 요청 구간
        //andDo : 응답 구간(결과가 나오고 나서 무언가를 하고 싶을
때)
        //andExpect : 검증 구간(결과를 기대하는 구간)

        //Create
    mockMvc
      .perform(post("/api/worlds")
      .content(docs::newEntity, "김길동"))
      .andDo(print())
      .andExpect(is2xx())
      .andDo(result(docs::context , "entity1"))
```

```
    ;

        //Read
    String url = docs.context("entity1", "$._links.self.hre
f");
    mockMvc
      .perform(post(url))
      .andDo(print())
      .andExpect(is2xx())
    ;
    mockMvc
      .perform(get(url))
      .andDo(print())
      .andExpect(is4xx())
    ;

        //Update
    Map<String,Object> body = docs.context("entity1");
        mockMvc
      .perform(put(url)
      .content(docs::updateEntity, body, "박길동"))
      .andExpect(is2xx())
      .andDo(print())
    ;

        //Delete
    mockMvc
      .perform(delete(url))
      .andDo(print())
      .andExpect(is2xx())
    ;
    }
}
```

- 이제 HelloTests.java와 WorldTests.java를 양쪽에 동시에 열어놓습니다.

```java
public class HelloTests {
  void contextLoad() throws Exception {

    //Create
    mockMvc
      .perform(post(uri: "/api/helloes")
      .content(docs::newEntity, t: "김길동"))
      .andDo(print())
      .andExpect(is2xx())
      .andDo(result(docs::context , key: "entity1"))
    ;

    //Read
    String url = docs.context(key: "entity1", jsonPath: "$._links.self.
    mockMvc
      .perform(post(url))
      .andDo(print())
      .andExpect(is2xx())
    ;
    mockMvc
      .perform(get(url))
      .andDo(print())
      .andExpect(is4xx())
    ;

    //Update
    Map<String,Object> body = docs.context(key: "entity1");
    mockMvc
      .perform(put(url)
      .content(docs::updateEntity, body, u: "박길동"))
      .andExpect(is2xx())
      .andDo(print())
    ;

    //Delete
    mockMvc
      .perform(delete(url))
```

```java
16  public class WorldTests {
25    void contextLoad() throws Exception {
45
46      //Create
47      mockMvc
48        .perform(post(uri: "/api/worlds")
49        .content(docs::newEntity, t: "김길동"))
50        .andDo(print())
51        .andExpect(is2xx())
52        .andDo(result(docs::context , key: "entity1"))
53      ;
54
55      //Read
56      String url = docs.context(key: "entity1", jsonPath: "$._links.self.href");
57      mockMvc
58        .perform(post(url))
59        .andDo(print())
60        .andExpect(is2xx())
61      ;
62      mockMvc
63        .perform(get(url))
64        .andDo(print())
65        .andExpect(is4xx())
66      ;
67
68      //Update
69      Map<String,Object> body = docs.context(key: "entity1");
70      mockMvc
71        .perform(put(url)
72        .content(docs::updateEntity, body, u: "박길동"))
73        .andExpect(is2xx())
74        .andDo(print())
75      ;
76
77      //Delete
78      mockMvc
79        .perform(delete(url))
```

- 먼저 아까 테스트했던 Read 부분으로 가줍니다.

  RestfulJpaRepository는 Read의 메소드가 POST입니다.

  하지만 JpaRepository는 Read의 메소드가 GET입니다.

- WorldTest.java의 //Read 부분을 아래 코드로 변경합니다.

```java
//Read
String url = docs.context("entity1", "$._links.self.href");
mockMvc
  .perform(get(url))
  .andDo(print())
  .andExpect(is2xx())
;
mockMvc
  .perform(post(url))
  .andDo(print())
  .andExpect(is4xx())
;
```

```java
//Read
String url = docs.context(key: "entity1", jsonPath: "$._links.self.href");
mockMvc
    .perform(get(url))
    .andDo(print())
    .andExpect(is2xx())
;
mockMvc
    .perform(post(url))
    .andDo(print())
    .andExpect(is4xx())
;
```

- HelloTest는 post를 2xx 번대로 예측한 반면 WorldTest는 get을 2xx 번대로 예측합니다.

  또한 HelloTest는 get을 4xx 번대로 예측한 반면 WorldTest는 post를 4xx 번대로 예측합니다.

- 2개의 코드 전부 Update와 Delete 부분은 주석 처리를 해줍니다.

  그 후 테스트를 실행하여 초록색 마크가 뜨는지 확인합니다.

  HelloTests.java는 위에 테스트를 하였기에 WorldTests.java의 결과만 사진으로 보여드리겠습니다.

- 아래 사진을 보시면 예상대로 GET으로 select를 하는걸 확인해보실 수 있습니다.

```
Hibernate:
    select
        w1_0.id,
        w1_0.email,
        w1_0.name
    from
        example_world w1_0
    where
        w1_0.id=?
2026-03-13T13:58:14.535+09:00 TRACE 15096 --- [example2] [           main] org.hibernate.orm.jdbc.bind             : binding parameter (1:BIGINT) <- [1]

http://localhost/api/worlds/1
----------------------------------------

MockHttpServletRequest:
      HTTP Method = GET
      Request URI = /api/worlds/1
       Parameters = {}
          Headers = []
             Body = null
    Session Attrs = {}

Handler:
             Type = org.springframework.data.rest.webmvc.RepositoryEntityController
           Method = org.springframework.data.rest.webmvc.RepositoryEntityController#getItemResource(RootResourceInformation, Serializable, PersistentEntityR
```

- 또한 POST 요청건은 404 에러를 띄우는걸 확인해보실 수 있습니다.

```
http://localhost/api/worlds/1
----------------------------------------

MockHttpServletRequest:
      HTTP Method = POST
      Request URI = /api/worlds/1
       Parameters = {}
          Headers = [Content-Type:"application/json;charset=UTF-8"]
             Body = {}
    Session Attrs = {}

Handler:
             Type = io.u2ware.common.data.jpa.webmvc.RestfulJpaRepositoryController
           Method = io.u2ware.common.data.jpa.webmvc.RestfulJpaRepositoryController#g
```

```
MockHttpServletResponse:
            Status = 404
     Error message = null
           Headers = [Vary:"Origin", "Access-Control-
      Content type = null
              Body =
     Forwarded URL = null
    Redirected URL = null
           Cookies = []
```

- 이로써 저희가 발견할 수 있는 차이점은 Read 시
  - RestfulJpaRepository는 Method가 POST,
  - JpaRepository는 Method가 GET이라는 점입니다.

- 다음으로 전체 목록을 불러오는 코드입니다.
  - RestfulJpaRepository는 search로 가져온다는 것을 배웠고
  - JpaRepository는 search를 안 붙이는 것을 알고 있습니다.

  테스트를 해보겠습니다.
- //Create 위에 각각 코드를 작성합니다.
  - HelloTests.java

    ```
    // RestfulJpaRepository
    mockMvc
      .perform(get("/api/helloes"))
      .andExpect(is4xx())
    ;
    ```

  - WorldTests.java

    ```
    // JpaRepository
    mockMvc
      .perform(get("/api/worlds"))
      .andExpect(is2xx())
    ;
    ```

- 2개의 코드를 동시에 보면 아래 사진과 같은 코드가 작성됩니다.

```java
@Test
void contextLoad() throws Exception {
  //perform : 요청 구간
  //andDo : 응답 구간(결과가 나오고 나서 무언가를 하고 싶을때)
  //andExpect : 검증 구간(결과를 기대하는 구간)

  // RestfulJpaRepository
  mockMvc
    .perform(get(uri: "/api/helloes"))
    .andExpect(is4xx())
  ;
  /*
  mockMvc
    .perform(get("/api/helloes/search"))
    .andExpect(is4xx())
  ;

  mockMvc
    .perform(post("/api/helloes/search"))
    .andExpect(is2xx())
  ;

  //Create
  mockMvc
    .perform(post("/api/helloes")
    .content(docs::newEntity, "김길동"))
    .andDo(print())
```

```java
24  @Test
25  void contextLoad() throws Exception {
26    //perform : 요청 구간
27    //andDo : 응답 구간(결과가 나오고 나서 무언가를 하고 싶을때)
28    //andExpect : 검증 구간(결과를 기대하는 구간)
29
30    // JpaRepository
31    mockMvc
32      .perform(get(uri: "/api/worlds"))
33      .andExpect(is2xx())
34    ;
35    /*
36    mockMvc
37      .perform(get("/api/worlds/search"))
38      .andExpect(is4xx())
39    ;
40
41    mockMvc
42      .perform(post("/api/worlds/search"))
43      .andExpect(is4xx())
44    ;
45
46    //Create
47    mockMvc
48      .perform(post("/api/worlds")
49      .content(docs::newEntity, "김길동"))
50      .andDo(print())
```

- JpaRepository는 get요청에 /search를 안 붙이면 정상적으로 목록을 불러오니 2xx 번대로 예측하고
- RestfulJpaRepository은 get요청에 /search를 안붙였으니 4xx번대로 예측합니다.

  테스트를 실행 후 2개 다 초록색 마크가 뜨는지 확인합니다.

```
Test Runner for Java
  HelloTests $(symbol-namespace) com.ubisam.projects.semina.ba
  contextLoad() $(symbol-class) HelloTests < $(symbol-namespace
> 9 older results
```

```
Test Runner for Java
  WorldTests $(symbol-namespace) com.ubisam.projects.s
  contextLoad() $(symbol-class) WorldTests < $(symbol-na
> 10 older results
```

- WorldTests의 Debug Console에는 Hibernate가 찍힌것도 확인해 보실 수 있습니다.

```
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Hibernate:
    select
        w1_0.id,
        w1_0.email,
        w1_0.name
    from
        example_world w1_0
    offset
        ? rows
    fetch
        first ? rows only
2026-03-13T14:12:34.315+09:00 TRACE 7324 --- [example2] [           main] org.hibernate.orm.jdbc.bind              : binding parameter (1:INTEGER) <- [0]
2026-03-13T14:12:34.315+09:00 TRACE 7324 --- [example2] [           main] org.hibernate.orm.jdbc.bind              : binding parameter (2:INTEGER) <- [20]
2026-03-13T14:12:34.512+09:00  INFO 7324 --- [example2] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for
2026-03-13T14:12:34.514+09:00  INFO 7324 --- [example2] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-03-13T14:12:34.516+09:00  INFO 7324 --- [example2] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.
```

- 반면 HelloTests의 Debug Console에는 Hibernate가 찍히지 않은 것도 확인해 보실 수 있습니다.

```
2026-03-13T14:14:10.314+09:00  INFO 26132 --- [example2] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-03-13T14:14:10.320+09:00  INFO 26132 --- [example2] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-03-13T14:14:10.320+09:00  INFO 26132 --- [example2] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.
```

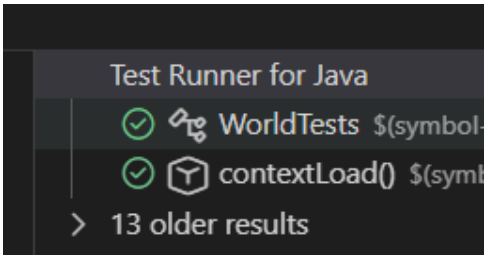
- 다음으로는 search에 대한 mockMvc 코드를 작성합니다.
  - HelloTests.java

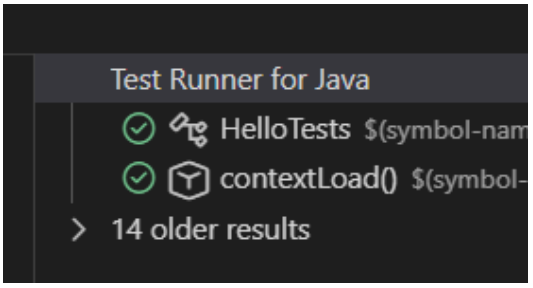
```
mockMvc
  .perform(get("/api/helloes/search"))
  .andExpect(is4xx())
;
```

  - WorldTests.java

```
mockMvc
  .perform(get("/api/worlds/search"))
  .andExpect(is4xx())
;
```

- HelloTests 같은 경우는 정상적으로 /search를 붙였으나 get요청이기에 4xx번대로 나올 것으로 기대하고
- WorldTests 같은 경우는 정상적으로 get요청이나 /search가 붙었기에 똑같이 4xx번대로 나올 것을 기대합니다.
  각각 테스트 후 초록색 마크를 확인합니다.
- 아래 사진을 통해 정상적으로 기대한 값이 나온 것을 확인해볼 수 있습니다.





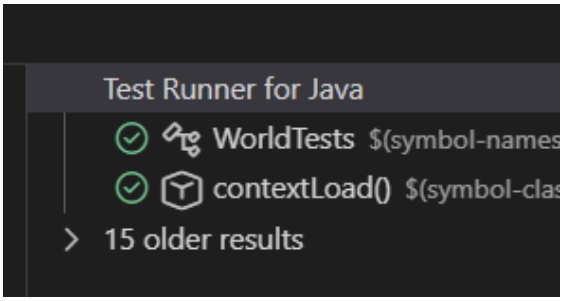
- 마지막으로 /search도 붙이고 post방식으로 요청하는 것을 테스트해보겠습니다.
  각각 아래 코드를 추가합니다.
  - HelloTests.java

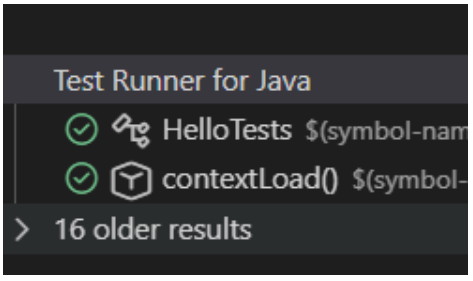
```
mockMvc
  .perform(post("/api/helloes/search"))
  .andExpect(is2xx())
;
```

- WorldTests.java

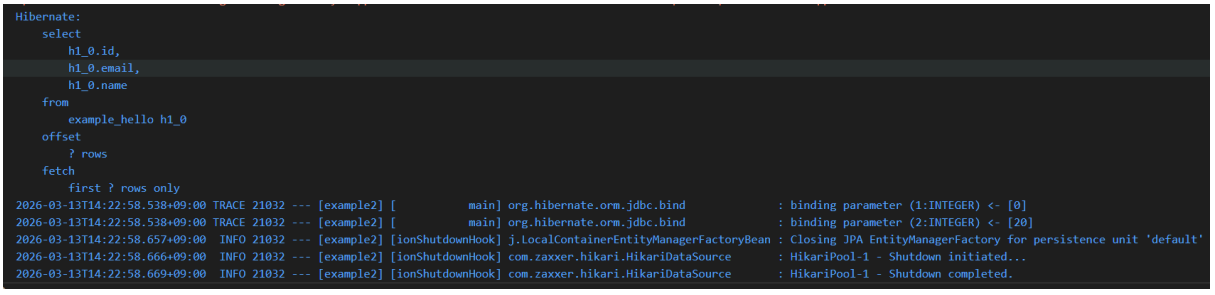
```
mockMvc
  .perform(post("/api/worlds/search"))
  .andExpect(is4xx())
;
```

- HelloTests 같은 경우는 정상적으로 /search를 붙였고 post요청이기에 2xx번대로 나올 것으로 기대하고
- WorldTests 같은 경우는 /search를 붙였고 post요청이기에 4xx번대로 나올 것을 기대합니다.
  각각 테스트 후 초록색 마크를 확인합니다.





- HelloTests 의 Debug Console에는 Hibernate가 찍힌것도 확인해 보실 수 있습니다.



```
Hibernate:
    select
        h1_0.id,
        h1_0.email,
        h1_0.name
    from
        example_hello h1_0
    offset
        ? rows
    fetch
        first ? rows only
2026-03-13T14:22:58.538+09:00 TRACE 21032 --- [example2] [           main] org.hibernate.orm.jdbc.bind              : binding parameter (1:INTEGER) <- [0]
2026-03-13T14:22:58.538+09:00 TRACE 21032 --- [example2] [           main] org.hibernate.orm.jdbc.bind              : binding parameter (2:INTEGER) <- [20]
2026-03-13T14:22:58.657+09:00  INFO 21032 --- [example2] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-03-13T14:22:58.666+09:00  INFO 21032 --- [example2] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-03-13T14:22:58.669+09:00  INFO 21032 --- [example2] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.
```

- 이로써 저희가 발견할 수 있는 2번째 차이점은 전체 목록을 불러올 시

- RestfulJpaRepository는 Method가 POST 이고 /search를 사용해야 한다는 점이고,
- JpaRepository는 Method가 GET이고 /search를 사용하지 말아야 한다는 점입니다.

- 최종적으로 전체 테스트를 실행한 후 마치겠습니다.

  터미널을 열어서 아래 사진과 같이 .\mvnw test 를 입력하여 테스트를 진행합니다.

```
PROBLEMS 5   OUTPUT   DEBUG CONSOLE   TERMINAL   TEST RESULTS   PORTS

PS C:\Users\user\Desktop\CHJ\personal\com.ubisam.projects.semina.backend> .\mvnw test
```

- 아래 사진과 같이 BUILD SUCCESS가 뜨면 통과입니다.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  18.506 s
[INFO] Finished at: 2026-03-13T14:28:21+09:00
[INFO] ------------------------------------------------------------------------
PS C:\Users\user\Desktop\CHJ\personal\com.ubisam.projects.semina.backend>
```

이렇게 세미나 추가 내용에 대한 공유였습니다.

제 소스 파일은 github에 올려놨습니다. 앞으로는 아래 리포지토리로 소스 파일을 올릴 예정입니다.

https://github.com/ubisam-heung/com.ubisam.projects.semina.backend

https://github.com/ubisam-heung/com.ubisam.projects.semina.frontend

긴 글 이였는데도 읽어주셔서 감사합니다 😊